МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

_____

(наименование университета)

ЛАБОРАТОРНАЯ РАБОТА № 1

Итерационные методы вычисления элементарных функций: квадратный корень, кубический корень и натуральный логарифм

Вариант 13

Дисциплина: Теория алгоритмов
Направление подготовки: 38.03.05 «Бизнес-информатика»
Курс: 2

Выполнил: _____ (ФИО), группа _____

Проверил: _____ (ФИО преподавателя)

20____ г.

1. Цель и задачи работы

Цель работы: сформировать представление об итерационных алгоритмах как о вычислительных процедурах, результат которых достигается последовательным уточнением приближений, и освоить приёмы оценки их сходимости и трудоёмкости.

Задачи:

- изучить понятия итерационного процесса, начального приближения, критерия остановки и скорости сходимости;
- реализовать вычисление √a и $\sqrt[3]{b}$ методом Ньютона и ln с разложением в степенной ряд;
- экспериментально определить число итераций, необходимое для достижения заданной точности ε;
- сравнить результаты с эталонными значениями модуля `math` и оценить абсолютную погрешность.

2. Задание варианта 13

Вычислить √31, $\sqrt[3]{100}$ и ln 0,2 с точностью $\varepsilon = 10^{-5}$.

- критерий остановки для корней — по разности соседних приближений: $|x_{n+1} - x_n| < \varepsilon$;
- для логарифма — по модулю члена ряда: $|t_k| < \varepsilon$;
- начальное приближение: $x_0 = a/2$ (для кубического корня $x_0 = b/2$);
- дополнительное требование Д1: вывести таблицу итераций для каждой функции (номер шага, x_n, $|x_{n+1} - x_n|$) и проанализировать изменение числа верных знаков от шага к шагу.

3. Краткие теоретические сведения

Итерационный алгоритм строит последовательность приближений

$$x_{n+1} = \varphi(x_n)$$

и определяется тремя элементами: начальным приближением x_0, итерационной формулой φ и критерием остановки. Результативность обеспечивается критерием остановки и ограничением N_{max} на число итераций.

Порядок сходимости. Если погрешность $e_n = |x_n - x^*|$ удовлетворяет условию

$$e_{n+1} \approx C\, e_n^p$$

то при p = 1 сходимость линейная (число верных знаков растёт на константу за шаг), при p = 2 — квадратичная (число верных знаков примерно удваивается).

Метод Ньютона для уравнения F(x) = 0:

$$x_{n+1} = x_n - \frac{F(x_n)}{F'(x_n)}$$

Квадратный корень ($F(x) = x^2 - a$, формула Герона):

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$$

Кубический корень ($F(x) = x^3 - b$):

$$x_{n+1} = \frac{1}{3}\left(2x_n + \frac{b}{x_n^2}\right)$$

Для b < 0 используется тождество $\sqrt[3]{b} = -\sqrt[3]{|b|}$.

Натуральный логарифм вычисляется рядом, сходящимся при любом c > 0:

$$\ln c = 2\left(y + \frac{y^3}{3} + \frac{y^5}{5} + \dots\right), \quad y = \frac{c-1}{c+1}$$

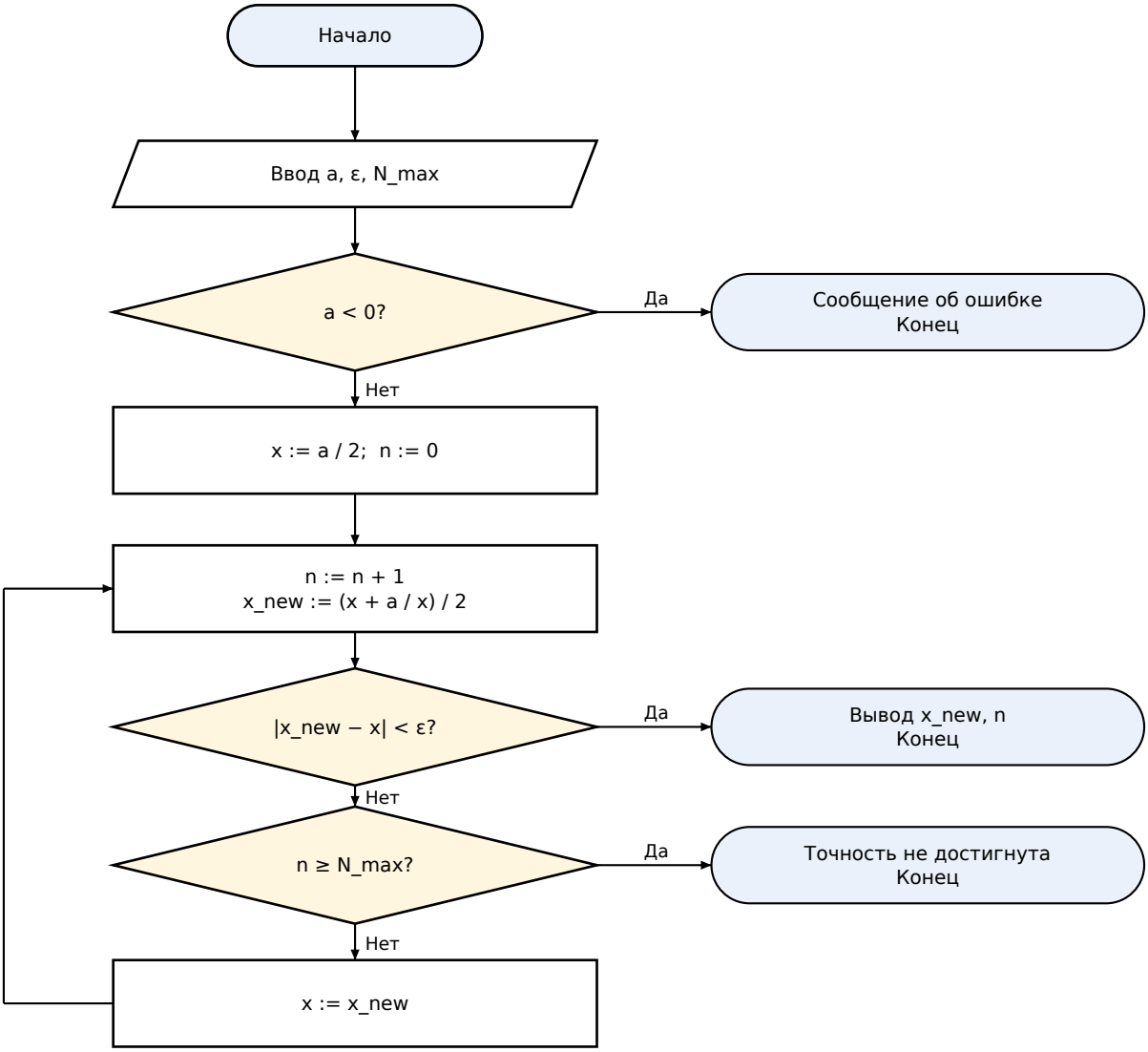
Каждый следующий член получается из предыдущего умножением степени на y^2:

$$t_k = \frac{2\,y^{2k+1}}{2k+1}$$

Для c = 0,2 имеем y = −2/3, |y| = 2/3.

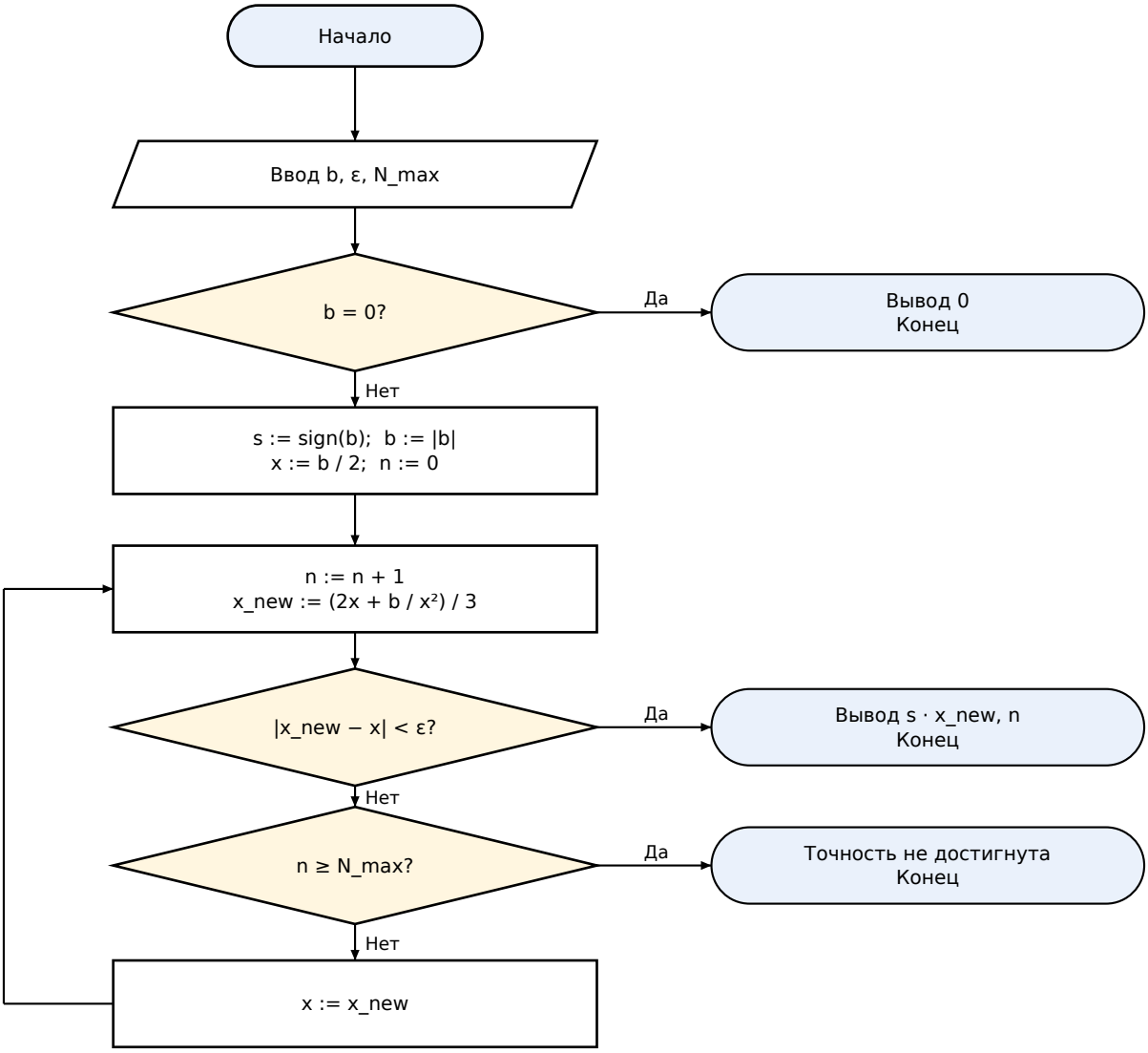
4. Блок-схемы алгоритмов

4.1. Вычисление квадратного корня

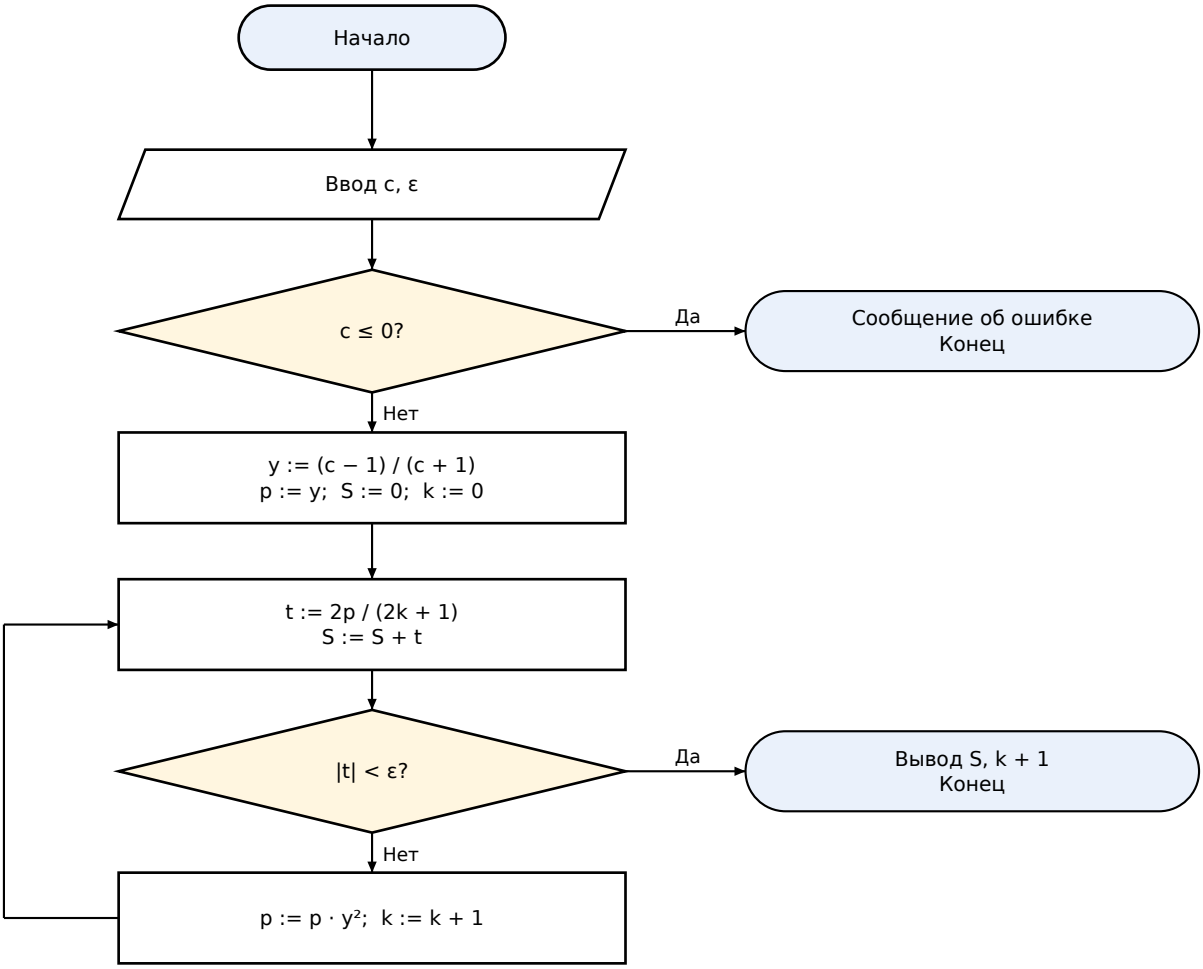


Случай a = 0 (результат 0, число итераций 0) обрабатывается в программе до входа в цикл.

4.2. Вычисление кубического корня



4.3. Вычисление натурального логарифма



5. Листинг программы

Файл `lab1_variant13.py` (Python 3.10+). Функции `math.sqrt`, `math.log` и `**0.5` внутри вычислительных функций не используются; модуль `math` нужен только для эталонных значений.

```
"""
Лабораторная работа № 1. Вариант 13.
Итерационные методы вычисления sqrt(a), cbrt(b), ln(c).
a=31, b=100, c=0.2, eps=1e-5, критерий остановки: по разности (P), x0 = a/2, доп. требование Д1.
"""
import math

A, B, C = 31, 100, 0.2
EPS = 1e-5
N_MAX = 100


def sqrt_newton(a, eps=EPS, n_max=N_MAX, trace=None):
    """sqrt(a) методом Ньютона (Герон). Остановка по разности |x_new - x| < eps."""
    if a < 0:
        raise ValueError("Подкоренное выражение должно быть неотрицательным")
    if a == 0:
        return 0.0, 0
    x = a / 2                               # x0 = a/2
    for n in range(1, n_max + 1):
        x_new = 0.5 * (x + a / x)
        if trace is not None:
            trace.append((n, x_new, abs(x_new - x)))
        if abs(x_new - x) < eps:
            return x_new, n
        x = x_new
    raise RuntimeError("Точность не достигнута за N_MAX итераций")


def cbrt_newton(b, eps=EPS, n_max=N_MAX, trace=None):
    """cbrt(b) методом Ньютона. Остановка по разности |x_new - x| < eps."""
    if b == 0:
        return 0.0, 0
    sign = -1 if b < 0 else 1
    b = abs(b)
    x = b / 2                               # x0 = b/2 (аналог x0 = a/2)
    for n in range(1, n_max + 1):
        x_new = (2 * x + b / (x * x)) / 3
        if trace is not None:
            trace.append((n, x_new, abs(x_new - x)))
        if abs(x_new - x) < eps:
            return sign * x_new, n
        x = x_new
    raise RuntimeError("Точность не достигнута за N_MAX итераций")


def ln_series(c, eps=EPS, n_max=10_000, trace=None):
    """ln(c) = 2*(y + y^3/3 + y^5/5 + ...), y=(c-1)/(c+1). Остановка по |член| < eps."""
    if c <= 0:
        raise ValueError("Аргумент логарифма должен быть положительным")
    y = (c - 1) / (c + 1)
    y2 = y * y
    power, total = y, 0.0
    for k in range(n_max):
        term = 2 * power / (2 * k + 1)
        if trace is not None:
            trace.append((k + 1, total + term, abs(term)))
        total += term                       # последний (малый) член тоже добавляем
        if abs(term) < eps:
            return total, k + 1
        power *= y2
    raise RuntimeError("Точность не достигнута за n_max итераций")


def correct_digits(x, ref):
    """Число верных десятичных знаков после запятой."""
    err = abs(x - ref)
    return 16 if err == 0 else max(0, int(-math.log10(err)))
```

```
def print_trace(title, trace, ref):
    print(f"\n{title}")
    print(f"{'n':>3}{'x_n':>18}{'|x_n - x_(n-1)|':>20}{'верных знаков':>16}")
    for n, x, d in trace:
        print(f"{n:>3}{x:>18.10f}{d:>20.3e}{correct_digits(x, ref):>16}")


if __name__ == "__main__":
    tables = {}
    rows = []
    for name, f, arg, ref in [("sqrt", sqrt_newton, A, math.sqrt(A)),
                              ("cbrt", cbrt_newton, B, B ** (1 / 3)),
                              ("ln", ln_series, C, math.log(C))]:
        tr = []
        val, n = f(arg, trace=tr)
        rows.append((name, arg, val, ref, abs(val - ref), n))
        tables[name] = (tr, ref)

    print(f"{'Функция':<8}{'Аргумент':>9}{'Результат':>16}{'Эталон':>16}"
          f"{'Погрешность':>14}{'Итераций':>10}")
    for name, arg, val, ref, err, n in rows:
        print(f"{name:<8}{arg:>9}{val:>16.10f}{ref:>16.10f}{err:>14.2e}{n:>10}")

    print_trace("Д1: sqrt(31)", *tables["sqrt"])
    print_trace("Д1: cbrt(100)", *tables["cbrt"])
    print_trace("Д1: ln(0.2) (x_n = частичная сумма, шаг = |член ряда|)", *tables["ln"])
```

6. Результаты расчётов

Таблица 1. Результаты (вариант 13, $\varepsilon = 10^{-5}$)

Функция	Аргумент	Результат	Эталон (math)	Абс. погрешность	Число итераций
√a	31	5,5677643628	5,5677643628	$8{,}88 \cdot 10^{-16}$	6
∛b	100	4,6415888336	4,6415888336	$1{,}28 \cdot 10^{-13}$	10
ln c	0,2	−1,6094325152	−1,6094379124	$5{,}40 \cdot 10^{-6}$	12

Таблица 2. Д1: итерации для √31

n	x_n	$\lvert x_n - x_{n-1} \rvert$	Верных знаков
1	8,7500000000	$6{,}750 \cdot 10^{0}$	0
2	6,1464285714	$2{,}604 \cdot 10^{0}$	0
3	5,5950039429	$5{,}514 \cdot 10^{-1}$	1
4	5,5678306715	$2{,}717 \cdot 10^{-2}$	4
5	5,5677643632	$6{,}631 \cdot 10^{-5}$	9
6	5,5677643628	$3{,}948 \cdot 10^{-10}$	15

Таблица 3. Д1: итерации для ∛100

n	x_n	$\lvert x_n - x_{n-1} \rvert$	Верных знаков
1	33,3466666667	$1{,}665 \cdot 10^{1}$	0
2	22,2610871255	$1{,}109 \cdot 10^{1}$	0
3	14,9079892640	$7{,}353 \cdot 10^{0}$	0
4	10,0886420142	$4{,}819 \cdot 10^{0}$	0
5	7,0532628641	$3{,}035 \cdot 10^{0}$	0
6	5,3722119659	$1{,}681 \cdot 10^{0}$	0
7	4,7364493295	$6{,}358 \cdot 10^{-1}$	1
8	4,6434759942	$9{,}297 \cdot 10^{-2}$	2
9	4,6415896005	$1{,}886 \cdot 10^{-3}$	6
10	4,6415888336	$7{,}669 \cdot 10^{-7}$	12

Таблица 4. Д1: итерации для ln 0,2

В столбце x_n указана частичная сумма ряда, в третьем столбце — модуль добавленного члена ряда.

n	x_n	$\lvert x_n - x_{n-1}\rvert$	Верных знаков
1	−1,3333333333	$1{,}333\cdot10^{0}$	0
2	−1,5308641975	$1{,}975\cdot10^{-1}$	1
3	−1,5835390947	$5{,}267\cdot10^{-2}$	1
4	−1,6002612842	$1{,}672\cdot10^{-2}$	2
5	−1,6060417942	$5{,}781\cdot10^{-3}$	2
6	−1,6081437978	$2{,}102\cdot10^{-3}$	2
7	−1,6089342949	$7{,}905\cdot10^{-4}$	3
8	−1,6092387827	$3{,}045\cdot10^{-4}$	3
9	−1,6093581896	$1{,}194\cdot10^{-4}$	4
10	−1,6094056731	$4{,}748\cdot10^{-5}$	4
11	−1,6094247670	$1{,}909\cdot10^{-5}$	4
12	−1,6094325152	$7{,}748\cdot10^{-6}$	5

7. Анализ результатов

1. √31. Число верных знаков по шагам: 0, 0, 1, 4, 9, 15. Начиная с 3-го шага оно примерно удваивается за итерацию — это квадратичная сходимость метода Ньютона. Достаточно 6 итераций.

2. ∛100. На шагах 1–6 верных знаков нет, затем их число растёт: 1, 2, 6, 12. Начальное приближение $x_0 = 50$ далеко от корня (≈ 4,64), и пока x_n велико, величина уменьшается лишь примерно в 1,5 раза за шаг (сходимость близка к линейной). Квадратичный режим наступает вблизи корня. Поэтому потребовалось 10 итераций против 6 для квадратного корня.

3. ln 0,2. Число верных знаков растёт медленно: 5 знаков после 12 членов ряда. При $|y| = 2/3$ члены убывают примерно в $y^2 \approx 0{,}44$ раза за шаг — это линейная сходимость.

4. Фактическая погрешность методов Ньютона ($\approx 10^{-16}$ и $\approx 10^{-13}$) на много порядков меньше ε: к моменту выполнения критерия по разности квадратичный процесс уже сошёлся с запасом. Для ряда погрешность ($5{,}4\cdot10^{-6}$) одного порядка с ε.

5. Критерий «по разности» надёжен для методов Ньютона из-за быстрой сходимости, но для линейно сходящегося ряда он может занижать реальную погрешность: остаток ряда способен превышать последний член. Поэтому в программе последний член добавляется к сумме перед выходом.

8. Выводы

1. Реализованы итерационные алгоритмы вычисления √a, ∛b (метод Ньютона) и ln c (степенной ряд). Все три достигли точности $\varepsilon = 10^{-5}$ за 6, 10 и 12 итераций соответственно.

2. Метод Ньютона сходится квадратично, ряд для логарифма — линейно; это подтверждается таблицами Д1.

3. Число итераций методов Ньютона существенно зависит от начального приближения: далёкое x_0 увеличивает число шагов на этапе, пока метод ещё не вышел в квадратичный режим.

9. Ответы на контрольные вопросы

1. **Дайте определение итерационного алгоритма.** Итерационный алгоритм получает результат как предел последовательности приближений $x_{n+1} = \varphi(x_n)$. Дискретность и детерминированность выполняются (шаги заданы однозначно), массовость — алгоритм работает для любых допустимых a, b, c. Результативность в общем случае не гарантируется конечностью процесса и обеспечивается критерием остановки и ограничением N_{max}.
2. **Вывод формулы Герона.** Для $F(x) = x^2 - a$ имеем $F'(x) = 2x$, поэтому $x_{n+1} = x_n - (x_n^2 - a)/(2x_n) = (x_n + a/x_n)/2$.
3. **Критерии «по разности» и «по невязке».** Первый проверяет, насколько сдвинулось приближение, второй — насколько x_n удовлетворяет уравнению $F(x_n) = 0$. Результаты могут различаться, если F очень пологая (малая невязка при большой ошибке по x) или очень крутая (малый шаг при большой невязке).
4. **Порядок сходимости.** Определяется соотношением $e_{n+1} \approx C \cdot e_n^{\,p}$. Метод Ньютона ($p = 2$) удваивает число верных знаков за шаг, а половинное деление ($p = 1$) лишь вдвое уменьшает интервал.
5. **Влияние начального приближения.** Чем дальше x_0 от корня, тем больше итераций. Пример: для $\sqrt[3]{100}$ при $x_0 = 50$ потребовалось 10 итераций, из них первые 6 не дали ни одного верного знака; для $\sqrt{31}$ при $x_0 = 15{,}5$ — 6 итераций.
6. **Ряд для ln(1+x) и ln 100.** Ряд для $\ln(1+x)$ сходится только при $-1 < x \le 1$, а для $\ln 100$ нужно $x = 99$ — ряд расходится. Ряд с $y = (c - 1)/(c + 1)$ даёт $|y| < 1$ при любом $c > 0$ и потому сходится.
7. **Нормализация $c = m \cdot 2^k$.** Приводит аргумент в область, где $|y| \le 1/3$, и ряд сходится быстрее; $\ln c = \ln m + k \cdot \ln 2$.
8. **Погрешность и ε.** Погрешность может быть намного меньше ε из-за быстрой сходимости (метод Ньютона). В линейно сходящихся процессах она может и превысить ε, так как остаток ряда может быть больше последнего члена.
9. **Зачем нужно N_{max}.** Ограничение защищает от зацикливания, если процесс не сходится или точность недостижима (например, из-за ошибок округления).
10. **Пример из экономики.** Расчёт внутренней нормы доходности (IRR) проекта — корень уравнения $NPV(r) = 0$ — находят методом Ньютона или секущих.

10. Список литературы

1. Кнут Д. Э. Искусство программирования. Т. 1. Основные алгоритмы. — 3-е изд. — М.: Вильямс, 2017.

2. Бахвалов Н. С., Жидков Н. П., Кобельков Г. М. Численные методы. — 9-е изд. — М.: Лаборатория знаний, 2020.